Write Verilog and Testbench Code to Implement 3 bit Synchronous UP/Down Counter

Aim

To design, implement, and simulate a **3-bit Up/Down Synchronous Counter** using Verilog HDL and verify its behavior across both counting modes using a testbench in Quartus Prime.

Objective:

- Understand the operation of a synchronous UP/DOWN counter.
- Design a 3-bit counter using Verilog HDL.
- Implement the design in Quartus Prime.
- Write a Verilog testbench for functional verification.
- Simulate the counter and observe its waveform.
- Verify UP counting, DOWN counting, reset, and clock synchronization.

Theory:

A **synchronous counter** updates all its internal flip-flops simultaneously because they share a single, **common external clock signal**. This unified clock pulse eliminates the cascading propagation delays typically found in asynchronous (ripple) counters.

A **3-bit counter** cycles through **8 distinct binary states** ($2^3 = 8$), starting from 000 up to 111. To control the direction of the count, an external mode control input (labeled **M** or **mode**) is introduced:

Up-Counting Mode (M = 0): The counter increments sequentially on every positive clock edge (000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000).

Down-Counting Mode (M = 1): The counter decrements sequentially on every positive clock edge 111 → 110 → 101 → 100 → 011 → 010 → 001 → 000 → 111).

Working

The 3-bit counter tracks state alterations on the rising edge of the clk signal. When active-low reset is pulled down (0), the counter clears immediately to 000. When reset is high (1), the counter evaluates the control input mode to shift the binary value forward or backward.

Counting Sequence

For UP counting:

000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000

For DOWN counting:

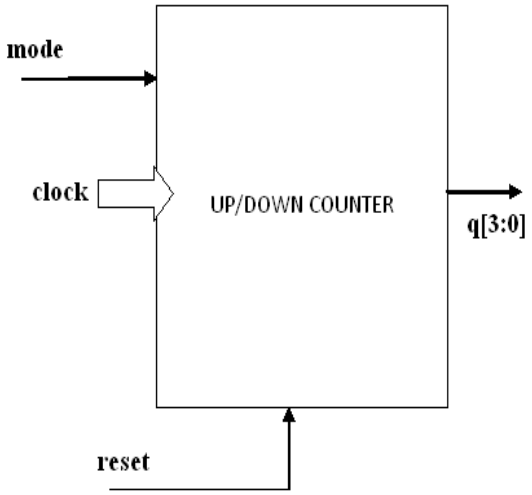
111 → 110 → 101 → 100 → 011 → 010 → 001 → 000 → 111

The counter can also have a reset input. When reset is activated, the counter is initialized to 000. In a synchronous design, the reset takes effect at the active clock edge.

The basic operation can be represented as: $Q_{next}: \begin{cases} 000 & if\ Reset = 1 \\ Q_1 + 1 & if UP/DOWN = 1 \\ Q_1 - 1 & if UP/DOWN = 0 \end{cases}$

where Q represents the present 3-bit counter output.

Block Diagram:-



Operation Table

Mode (M)	Current State ($Q_2Q_1Q_0$)	Next State ($Q_2^+Q_1^+Q_0^+$)	Operation
0	000	001	Up Count
0	111	000	Up Count (Roll-over)
1	111	110	Down Count
1	000	111	Down Count (Roll-under)

Algorithm

- Start the program.
- Define clk, reset, and direction control input mode.
- Define a 3-bit output vector count to monitor the states.
- Establish an edge-triggered block sensitive to the rising edge of the clock (posedge clk) or the falling edge of the reset (negedge reset).
- Evaluate reset: If active, force the count vector to 3'b000.
- Evaluate mode: If 0, increment count by 1. If 1, decrement count by 1.
- Apply sequential input sequences and toggle the clock within the testbench.

- Check the output transitions against the expected state table.
- Stop

Verilog Code (Design Module)

```
module sync_updown_counter_3bit (
    input clk,
    input reset,
    input mode, // 0 = Up, 1 = Down
    output reg [2:0] count
);

always @(posedge clk or negedge reset) begin
    if (!reset) begin
        count <= 3'b000;
    end else begin
        if (mode == 1'b0)
            count <= count + 1'b1; // Up counting
        else
            count <= count - 1'b1; // Down counting
    end
end

endmodule
```

Testbench

```
module sync_updown_counter_3bit_tb;
    reg clk;
    reg reset;
    reg mode;
    wire [2:0] count;

    // Instantiate Device Under Test (DUT)
    sync_updown_counter_3bit DUT (
        .clk(clk),
        .reset(reset),
        .mode(mode),
        .count(count)
    );
```

```
    // Generation of Clock signal (Period = 10ns)
    always #5 clk = ~clk;

    initial begin
        // Initialize Inputs
        clk = 0;
        reset = 0;
        mode = 0;
        #12;

        // Release reset to start UP counting
        reset = 1;
        #90; // Let it cycle through all 8 up states (0 to 7)

        // Change mode to DOWN counting
        mode = 1;
        #90; // Let it cycle through down states (7 down to 0)

        // Apply reset during runtime to verify safety clear
        reset = 0;
        #10;

        $stop; // End Simulation
    end
endmodule
```

Simulation Procedure (Quartus Prime & Questa Sim)

1. Open Intel Quartus Prime and launch the **New Project Wizard**.
2. Name the project and append both files: sync_updown_counter_3bit.v and sync_updown_counter_3bit_tb.v.
3. Mark sync_updown_counter_3bit as your **Top-Level Design Entity**.
4. Run compilation via **Processing → Start Compilation** and verify zero errors.
5. Open your simulator (Questa Sim / ModelSim) and compile the Verilog

6. source files.
7. Set sync_updown_counter_3bit_tb as the simulation top-level target.
8. Push clk, reset, mode, and count to the Wave window.
9. Run your simulation for at least **200 ns** to capture both modes.
10. Analyze wave patterns and match state vectors against your target parameters.

Expected Simulation Result

Time Frame	**Reset (reset)**	**Mode (mode)**	**Counter Output Vector (**count[2:0])	**Operation Observed**
0 - 12 ns	0	0	000	Asynchronous Reset Active
12 - 25 ns	1	0	001	First Up Count Item
…..	1	0	Increments up to 111, then wraps to 000	Full Up-Count Chain
102 ns	1	1	Decrements down from current state	Down-Count Inversion
…..	1	1	Decrements down to 000, then wraps to 111	Full Down-Count Chain
End Frame	0	1	000	Immediate clear action

Observation

The 3-bit Up/Down Synchronous Counter was designed and verified using Verilog HDL. Through the testbench implementation, the outputs were observed under both mode states. The counter updated states simultaneously at the positive clock edge without any step-delays, validating synchronous execution.

Applications

- Digital clocks, timers, and stopwatches.
- Analog-to-Digital Converters (ADC) using tracking methodologies.
- Memory addressing controls inside FIFO queues.
- Up/down frequency dividers and tracking odometers.
- Command execution sequence steps within processing units.

Advantages

- Minimal propagation delays because all flip-flops toggle together.
- Higher operating speeds compared to asynchronous/ripple counters.
- Direct control layout for directional changes via a single input line.
- Scaleable construction for larger bit widths (4-bit, 8-bit, etc.).

Conclusion

The 3-bit Up/Down Synchronous Counter was designed and verified using Verilog HDL. Through the testbench implementation, the outputs were observed under both mode states. The counter updated states simultaneously at every positive edge of the clock signal, confirming its synchronous behavior and eliminating cumulative propagation delays. The simulation waveforms successfully matched the theoretical state transitions for both up-counting and down-counting modes, proving the functional correctness of the digital design.

Viva Questions

1. What distinguishes a synchronous counter from an asynchronous counter?
2. How many states can a 3-bit counter navigate, and what is its maximum value?
3. What happens if the mode selection input changes values mid-way through a clock period?
4. What is the fundamental utility of using the non-blocking (<=) assignment operator inside synchronous always blocks?
5. How can you modify the code to turn this design into a Modulo-6 up/down counter?